ABC Engineering Services

Application Design

Supplier Invoice & Payment Management System

Table of Contents

1. High-Level Overview of the Business and Problem....3
2. Business Objectives for the System....3
3. Scope....3
   3.1 In Scope....3
   3.2 Out of Scope....4
4. System Actors / Users....4
5. Functional Requirements....5
   5.1 Supplier Management....5
   5.2 Invoice Management....5
   5.3 Payment Management....6
   5.4 Invoice Status and Financial Information....6
6. Business Rules....6
7. Business Processes....7
   Invoice Cancellation....7
8. Non-Functional Requirements....8
9. Data/Domain Model....8
   9.1 Domain Entities....8
   9.2 Define Relationships....8
   9.3 Define Attributes....9
      9.3.1 Supplier....9
      9.3.2 Invoice....9
      9.3.3 Payment....9
   9.4 Derived Information / Domain Behaviour....10
      9.4.1 Invoice Outstanding Amount....10
      9.4.2 Invoice Payment Status....10
      9.4.3 Overdue Invoices....10
   9.5 Domain Model Diagram....11
10. Technical / Solution Design....12
   10.1 High-Level Architecture....12
      10.1.1 Monolithic Architecture....12
      10.1.2 Layered Architecture....12
      10.1.3 REST API....12
      10.1.4 Persistence....13
   10.2 Application Structure....15
   10.3 API Design....15
      10.3.1 API Conventions....15
      10.3.2 API Endpoints....16
   10.4 Database Design....16
   10.5 Testing Approach....18
      10.5.1 Unit Testing....18
      10.5.2 Integration and End-To-End Testing....18

1. High-Level Overview of the Business and Problem

ABC Engineering Services Ltd. is a small company providing engineering, maintenance, and installation services to commercial customers. It currently employs approximately 50 staff, including engineers and business/support staff. It purchases materials, equipment, and services from external suppliers.

The business currently records supplier invoices and payments using spreadsheets. However, the business has grown considerably over recent years and so the number of suppliers and invoices it deals with has grown correspondingly. The Accounts department are finding it increasingly difficult to effectively process invoices and therefore maintain an accurate view of which invoices have been paid, which are due for payment, and if any are overdue.

ABC Service's directors have therefore decided that the process needs updating and wish to build a simple internal web application to centralise its supplier invoice and payment information. The system should allow staff to record and manage invoices and payments and provide a clear view of paid, outstanding, and overdue invoices.

The application is intended for internal business use and will not be accessed by suppliers or customers.

2. Business Objectives for the System

ABC Engineering Services Ltd want to:

1. **Centralise supplier invoice information**
   Replace the current spreadsheet-based approach with a centralised system for recording supplier invoices, thereby increasing the efficiency and reliability of the process.
2. **Record supplier payments**
   Maintain accurate records of payments made against supplier invoices.
3. **Identify outstanding invoices**
   Allow users to produce clear, accurate views of unpaid invoices.
4. **Identify outstanding invoices**
   Enable Accounts staff to easily identify invoices with outstanding balances.
5. **Improve visibility of amounts owed**
   Provide an accurate view of the company's outstanding liabilities to suppliers.

3. Scope

3.1 In Scope

The application will:

- Maintain a list of suppliers.
- Record and maintain supplier invoices.
- Support UK-based suppliers and invoices denominated in British pounds sterling only (in Phase 1).
- Record payments made against supplier invoices.
- Calculate and display the outstanding amount on an invoice.
- Determine the invoice payment status and identify overdue invoices.
- Search/filter invoices to identify outstanding and overdue items.
- Provide internal users with an overview of the company's outstanding supplier liabilities.

3.2 Out of Scope

The application will not attempt to provide a complete accounting solution or financial management system. Specifically it will not cover:

- Customer invoicing or accounts receivable.
- Payroll.
- Inventory or stock management.
- Purchasing or procurement management.
- General ledger or double-entry accounting.
- Tax/VAT reporting or submissions.
- Actual processing of payments through a bank or payment service.
- Supplier or customer access to the system.

4. System Actors / Users

Accounts User

The primary internal user of the application, responsible for:

- Creating and updating supplier records.
- Recording supplier invoices.
- Recording payments.
- Reviewing invoices and payments.
- Monitoring outstanding and overdue invoices.

5. Functional Requirements

5.1 Supplier Management

FR-01 — Create Supplier
The system shall allow the Accounts User to create a new supplier record.

FR-02 — View Supplier
The system shall allow the Accounts User to view supplier information.

FR-03 — View All Suppliers
The system shall allow the Accounts User to view information for all suppliers.

FR-04 — Update Supplier
The system shall allow the Accounts User to update supplier information.

FR-05 — Deactivate Supplier
The system shall allow the Accounts User to deactivate a supplier no longer used.

FR-06 — Reactivate Supplier
The system shall allow the Accounts User to reactivate a previously deactivated supplier.

5.2 Invoice Management

FR-07 — Record Supplier Invoice
The system shall allow the Accounts User to record a new supplier invoice, including the supplier, invoice number, invoice date, due date and invoice amount.

FR-08 — View Invoice
The system shall allow the Accounts User to view the details of a supplier invoice, including its payments, outstanding amount and current financial status.

FR-09 — Update Invoice
The system shall allow the Accounts User to update supplier invoice information, subject to applicable business rules.

FR-10 — Record Invoice Cancellation
The system shall allow the Accounts User to record that a supplier has cancelled an invoice. The User will enter the reason for cancellation and the system shall record the date and time at which the cancellation has been recorded.

5.3 Payment Management

FR-11 — Record Payment
The system shall allow the Accounts User to record a payment against a supplier invoice, including the payment date, amount, and payment reference.

FR-12 — View Payment
The system shall allow the Accounts User to view the details of a particular payment.

FR-13 — Record Payment Cancellation
The system shall allow the Accounts User to record the cancellation of a payment. The User shall provide a reason for cancellation, and the system shall record the date and time of the cancellation automatically.

5.4 Invoice Status and Financial Information

FR-14 — Calculate Outstanding Amount
The system shall calculate the outstanding amount for each supplier invoice based on the invoice amount and non-cancelled payments.

FR-15 — Determine Payment Status
The system shall determine an invoice's payment status as Outstanding, Partially Paid or Paid in Full, based on the invoice amount and payments recorded against the invoice that have not been cancelled.

6. Business Rules

BR-01 — Supplier Deactivation
Suppliers that are no longer used may be deactivated, rather than permanently deleted, and their details will be retained for archival purposes. A supplier shall not be deactivated while it has any non-cancelled invoice with an outstanding amount greater than zero.

BR-02 — Multiple Payments
An invoice may have multiple payments recorded against it.

BR-03 — Outstanding Amount
An invoice's outstanding amount shall be calculated as the invoice amount minus the total of payments made.

BR-04 — Payment Status
An invoice shall be considered:

Outstanding when no active payments have been made.
Partially Paid when active payments have been recorded but an outstanding balance remains.
Paid in Full when the outstanding balance is zero.

BR-05 — Overdue Invoices
An invoice shall be considered overdue when its due date has passed and it has an outstanding balance, unless the invoice has been cancelled.

BR-06 — Invoice Cancellation
An invoice shall be cancelled when the supplier has agreed to its cancellation. The date & time and reason for the cancellation shall be recorded. The cancelled invoice shall be retained for archival purposes and excluded from outstanding or overdue invoice calculations. Invoice cancellation shall be irreversible.

BR-07 — Invoice Cancellation with Payments
An invoice with associated payments shall only be cancelled after all such associated payments have been cancelled. Cancelling an invoice shall not automatically cancel its associated payments.

BR-08 — Payment Cancellation
A cancelled payment shall be retained for archival purposes but excluded from calculations of payments made and outstanding amounts for invoices. The reason for payment cancellation and the time and date at which it occurred shall be recorded.

BR-09 — Payment Correction
An active payment shall not be edited. If a payment has been recorded incorrectly, it shall be cancelled and a new payment may be recorded with the correct information.

7. Business Processes

Invoice Cancellation

1. Supplier agrees that the invoice can be cancelled.
2. Accounts User checks whether the invoice has associated payments.
3. Any associated payments are cancelled using the payment cancellation function.
4. Accounts User cancels the invoice.
5. The system records the cancellation date/time and reason.

6. The cancelled invoice remains in the system for archival purposes and is excluded from outstanding/overdue calculations.
7. Any financial restitution is handled outside this system or by another system.

8. Non-Functional Requirements

NFR-01 — Data Integrity
The system shall maintain consistent and accurate relationships between suppliers, invoices and payments, including when payments are recorded or cancelled.

NFR-02 — Consistency
The API shall use consistent resource naming, HTTP methods, and HTTP response conventions.

NFR-03 — Maintainability
The application shall use a clear separation of responsibilities and established Spring conventions to support future maintenance and enhancement.

NFR-04 — Testability
Key business rules and application functionality shall be covered by automated tests.

9. Data/Domain Model

9.1 Domain Entities

The Phase 1 domain entities are:

- Supplier
- Invoice
- Payment

System User is an actor rather than a domain entity and is therefore excluded from the Phase 1 data model. Authentication and user management may be considered as future functionality.

9.2 Define Relationships

Supplier–Invoice: A supplier may have many invoices. Each invoice must be associated with one supplier.

Invoice–Payment: An invoice may have zero or more payments. Each payment must be associated with one invoice.

9.3 Define Attributes

9.3.1 Supplier

Attribute	Required	Definition
Supplier Name	Yes	Uniquely identifies supplier
Business Address	Yes	Supplier's primary business address
Contact Name	No	Primary contact person
Contact Email	No	Email address of primary contact
Contact Telephone	No	Telephone number of primary contact
Active/Inactive	Yes	Is company currently using supplier

9.3.2 Invoice

Attribute	Required	Definition
ID	Yes	Uniquely identifies invoice
Supplier	Yes	The supplier the invoice relates to
Supplier Invoice Reference	Yes	Supplier's invoice number
Due Date	Yes	The date by which full payment is due
Amount	Yes	The total amount payable for the invoice
Cancellation Date and Time	No	Empty for valid invoices; for cancelled invoices, the date and time at which ABC recorded the cancellation.
Cancellation Reason	No	The reason for the cancellation.

9.3.3 Payment

Attribute	Required	Definition
ID	Yes	Uniquely identifies the payment
Invoice	Yes	The invoice the payment relates to
Payment Date	Yes	The date on which the payment was made

Payment Amount	Yes	The amount paid
Payment Method	Yes	The method used for payment
Payment Reference	No	A reference to identify the payment, e.g. bank/payment reference
Cancellation Date and Time	No	Empty for valid payments; for cancelled payments, the date and time at which ABC recorded the cancellation
Cancellation Reason	No	The reason for the cancellation

9.4 Derived Information / Domain Behaviour

9.4.1 Invoice Outstanding Amount

This will be calculated as the Invoice Amount less the total of payments made against the invoice.

9.4.2 Invoice Payment Status

This will be derived from the payments and the balance on the Invoice:

Situation	**Status**
No payments made	**Outstanding**
Payments made AND outstanding balance greater than zero	**Partially Paid**
Outstanding balance is zero	**Paid in Full**

9.4.3 Overdue Invoices

An invoice will be deemed overdue when all of the following conditions apply:

- The current date is beyond the due date.
- The outstanding amount is greater than zero.
- The invoice has not been cancelled.

9.5 Domain Model Diagram

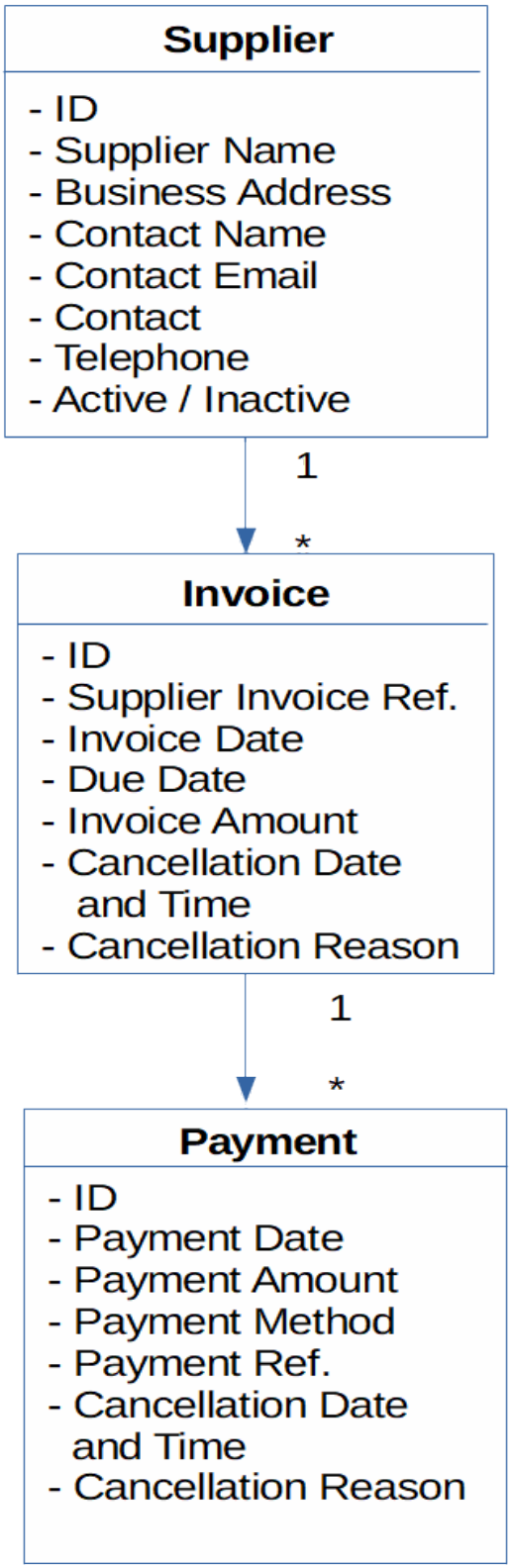


Figure 1: Phase 1 Domain Model

10. Technical / Solution Design

10.1 High-Level Architecture

10.1.1 Monolithic Architecture

The application will use a monolithic architecture, with the application components forming a single deployable system.

While a microservices architecture could be advantageous if this were a larger, more complex system – due to the options for independent deployment and scaling of individual services – these benefits are not required by the application in Phase 1, and adopting microservices would introduce unnecessary complexity.

Therefore, a monolithic architecture is considered the most appropriate for the small scope and anticipated scale of the application in Phase 1, and provides a simple operational and development model.

10.1.2 Layered Architecture

The application will use a layered architecture with clear separation of responsibilities between the main components. The responsibilities of the components are:

- **Controllers**: handle HTTP requests and responses, and provide the REST API interface.
- **Services**: coordinate application and business operations, and contain relevant business logic.
- **Repositories**: handle data persistence and interaction with the database.
- **Entities**: represent the core domain objects used by the application.

This separation will help to clearly define responsibilities, and support maintainability and testing.

10.1.3 REST API

The application will expose a RESTful HTTP API through which clients can interact with the system. The API will provide operations corresponding to the application's core functionality, including:

- Managing suppliers
- Managing invoices
- Managing payments

- Retrieving invoice information
- Retrieving payment information

The API will provide a clear boundary between the application and any client interface that may be developed.

Phase 1 will not include a web-based user interface. The initial implementation will focus on the REST API and backend functionality, allowing the core application and business rules to be developed and tested independently of a particular user interface. A web UI may be considered in a future phase if there is sufficient benefit in developing one.

10.1.4 Persistence

PostgreSQL will be used as the application's relational database, providing a mature and widely-used open-source database platform. JPA/Hibernate will provide object-relational mapping between the application's domain model and the database.

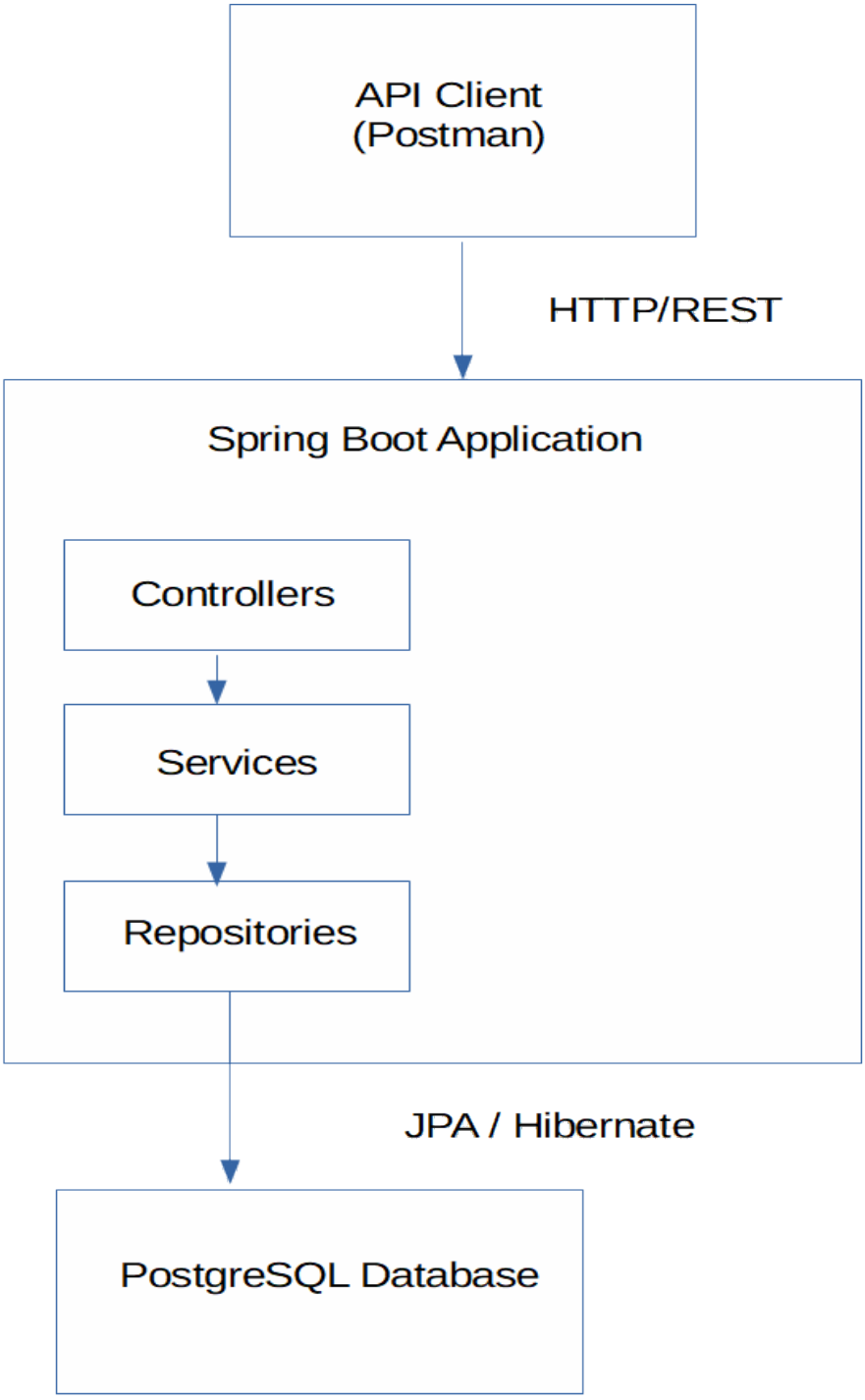


Figure 2: High-Level Application Architecture

10.2 Application Structure

Controllers

Controllers will handle incoming HTTP requests and responses, and delegate application operations to the relevant service, avoiding dealing directly with business logic.

Services

Services will coordinate application operations and business processes, including interactions between domain entities and repositories.

Repositories

Repositories will persist domain entities and provide access to data stored in the database.

Entities

Entities will represent the core domain objects – Supplier, Invoice, and Payment – and their relationships. Where appropriate, entities may also contain behaviour relating directly to their own domain data.

10.3 API Design

10.3.1 API Conventions

The application will expose a RESTful API providing access to the main resources of the system: Suppliers, Invoices and Payments. API operations will use standard HTTP methods and resource-oriented URLs.

The API will support the creation, retrieval, modification and appropriate removal or cancellation of these resources. Standard HTTP methods will be used according to the nature of each operation, with PATCH used where an operation represents a partial change to an existing resource, such as recording a cancellation.

Payments will be associated with their corresponding invoices, with the API allowing payments to be retrieved and recorded in the context of an invoice. The API design will reflect the relationships between the domain entities established in the data model.

The API will use appropriate HTTP response status codes to indicate the outcome of requests.

10.3.2 API Endpoints

Resource	Method	Endpoint	Purpose
Supplier	GET	/api/suppliers	List suppliers
Supplier	POST	/api/suppliers	Create supplier
Supplier	GET	/api/suppliers/{id}	View supplier
Supplier	PUT	/api/suppliers/{id}	Update supplier
Supplier	DELETE	/api/suppliers/{id}	Deactivate supplier
Supplier	PATCH	/api/suppliers/{id}	Reactivate supplier
Invoice	GET	/api/invoices	List invoices
Invoice	POST	/api/invoices	Create invoice
Invoice	GET	/api/invoices/{id}	View invoice
Invoice	PUT	/api/invoices/{id}	Update invoice
Invoice	PATCH	/api/invoices/{id}	Record invoice cancellation
Payment	GET	/api/payments/{id}	View payment
Payment	POST	/api/invoices/{id}/payments	Record a payment against an invoice
Payment	PATCH	/api/invoices/{id}/payments/{paymentId}	Record payment cancellation

10.4 Database Design

The application will use a relational database consisting of Supplier, Invoice, and Payment tables. The tables reflect relationships outlined in the domain model, with primary and foreign keys used to maintain referential integrity. Database constraints will be used where appropriate to enforce data integrity; however, application-level validation will be used for input and to ensure business rules are followed.

Supplier

Column	Data Type	Nullable	Key	Description
id	integer	No	Primary	Unique identifier
company_name	text	No		Business name of company
company_address	text	No		Primary business address

contact_name	varchar(100)	Yes		Primary contact at company
contact_email	varchar(254)	Yes		Email address of primary contact
contact_telephone	varchar(30)	Yes		Telephone number of primary contact
active	boolean	No		Is company currently actively supplying to ABC or not?

Invoice

Column	Data Type	Nullable	Key	Description
id	integer	No	Primary	Uniquely identifies invoice
supplier_id	integer	No	Foreign	Unique identifier of supplier invoice relates to
supplier_invoice_ref	varchar(100)	No		Reference supplier assigns to the invoice
invoice_date	date	No		Date the invoice was issued
due_date	date	No		Date payment is due
invoice_amount	numeric(10,2)	No		Invoice amount in GBP
cancelled_at	timestamp	Yes		Null means invoice active; otherwise stores data and time ABC record cancellation
cancellation_reason	Text	Yes		If cancelled, reason for cancellation; otherwise null

Payment

Column	Data Type	Nullable	Key	Description
id	integer	No	Primary	Uniquely identifies the payment
invoice_id	integer	No	Foreign	Unique identifier of the invoice payment relates to

payment_date	date	No		Date of payment
payment_amount	numeric(10,2)	No		Payment amount in GBP
payment_method	varchar(30)	No		Method used to make the payment
payment_ref	varchar(50)	No		Reference assigned to the payment by ABC; must be unique
cancellation_at	timestamp	Yes		Null means payment is active; otherwise stores date and time ABC record cancellation
cancellation_reason	Text	Yes		If cancelled, reason for cancellation; otherwise null

10.5 Testing Approach

10.5.1 Unit Testing

JUnit 6 is used for unit testing, with Mockito for mocking dependencies and MockMvc for testing controller behaviour. Tests cover the repository, service and controller layers, verifying individual components, their interactions, and key business rules.

Key business rules include:

- Calculation of outstanding amounts.
- Determination of payment status.
- Exclusion of cancelled invoices and payments from the appropriate calculations.
- Enforcement of supplier, invoice, and payment business rules.

10.5.2 Integration and End-To-End Testing

Integration and end-to-end testing is performed against the running Spring Boot application using REST API requests. PostgreSQL is used as the application database, with database state verified through pgAdmin.

Testing covers the Supplier, Invoice and Payment features, including normal operations, validation and relevant error conditions.